Redux and Redux Took Kit

Why we need Redux

The Problem — "Prop Drilling"

In React, data flows **parent → child** via props. When many components need the same data, you have to pass props through every level — even components that don't need it.

```
App
 └── Parent
      └── Child
           └── GrandChild  ← needs the data
```

You'd pass props through **every level** just to reach GrandChild. This is **prop drilling** and it gets messy fast.

Real Problems Without Redux

- **Prop drilling** — passing data through many unnecessary layers
- **Hard to share state** between sibling components
- **Difficult to debug** — hard to track where state changed
- **Scattered state** — state lives in many different components
- **Inconsistent data** — different parts of app show different values

---

What Redux Does

Redux creates a **single central store** that holds your entire app's state.

```
        Redux Store (single source of truth)
             ↓            ↓            ↓
        Header       Dashboard     Sidebar
```

Any component can **directly access** the store — no prop drilling needed.

In Short

> *Redux = **One place to store, one place to update, every component can access.***

It brings **predictability, consistency, and easier debugging** to your app's state.

Why We Need Redux Toolkit?

The Problem — "Vanilla Redux is Painful"

Redux is great, but writing it the traditional way has a lot of **boilerplate code** (repetitive, lengthy setup).

Problems With Vanilla Redux

1. Too Much Code for Simple Things

```
// Action Type
const INCREMENT = 'INCREMENT';

// Action Creator
const increment = () => ({ type: INCREMENT });

// Reducer
const reducer = (state = 0, action) => {
  switch (action.type) {
    case INCREMENT: return state + 1;
    default: return state;
  }
};
```

Just to increment a counter — **so much code!**

---

2. Too Many Files

You had to create separate files for:

- Action Types
- Action Creators
- Reducers
- Store setup

3. Immutability Was Hard

You had to manually return new state every time:

```
// Old way - easy to make mistakes
return { ...state, user: { ...state.user, name: action.payload } };
```

4. Async Logic Was Complex

Needed extra libraries like `redux-thunk` or `redux-saga` just to make API calls.

5. Store Setup Was Complicated

```
// Old Redux store setup
const store = createStore(
  rootReducer,
  applyMiddleware(thunk, logger)
);
```

What Redux Toolkit Solves

Problem	Vanilla Redux	Redux Toolkit
Boilerplate	Too much	Minimal
Immutability	Manual	Auto (uses Immer)
Async calls	Extra setup	Built-in (`createAsyncThunk`)
Store setup	Complex	Simple (`configureStore`)
File structure	Many files	One slice file

Same Counter in Redux Toolkit

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: {
    increment: (state) => state + 1,
  }
});
```

Much cleaner! Action types, action creators, and reducers — all in one place.

In Short

> *Redux Toolkit = **Redux without the pain.***

It is now the **official recommended way** to write Redux. There is no reason to use Vanilla Redux in a new project.

Architecture of Redux Toolkit (RTK)

This document outlines the step-by-step data flow in a Redux Toolkit application, from a user interaction to the final UI update.

1. UI Layer (The Trigger)

- The process begins in the **UI Layer**.
- A user interacts with a component, such as clicking an "Add to Cart" button.
- This interaction triggers an event (e.g., `onClick`).

2. Event Handler & Action Dispatch

- **Event Handler:** The application captures the event triggered by the user.
- **Action:** An action object is created, representing what needs to happen (e.g., the `AddtoCart` action).
- **Dispatch:** The event handler uses an **Action Dispatcher** to send this action from the UI layer to the Redux Store.

3. Redux Store (Centralized Storage)

- The action arrives at the **Redux Store**, which acts as the centralized storage for the application's global state (e.g., Cart data, Theme settings like Light/Dark).
- There is only **one** centralized Redux store in the application.
- The store holds the current state and delegates the incoming action to the appropriate Reducer.

4. Reducers (State Modification Logic)

- The store contains multiple **Reducers**. Each reducer is a function responsible for performing specific tasks and managing a specific slice of the state.
- **Action to Reducer:** The dispatched action is matched with its corresponding reducer function.
- **Update Logic:** The reducer executes the specific logic defined for that action (e.g., calculating the new cart total, changing `cart: 0` to `cart: 1`).

5. State Update

- After the reducer executes its logic, it returns the new state.
- The **Redux Store is updated** with these new values.

6. UI Update

- The UI Layer is subscribed to the Redux Store.
- Once the state in the store is updated, it triggers a **UI Update**.
- The UI re-renders to reflect the newly updated state (e.g., the cart counter on the screen visually changes from 0 to 1).

Step	Component	Role
1	UI Layer	User clicks a button (Event).
2	Action	A description of the change is created and dispatched.
3	Store	The central hub that holds the app's data.
4	Reducer	The logic that calculates the new state based on the action.
5	UI Update	The screen re-renders to show the new data.

Redux Slice

A **slice** in Redux (specifically Redux Toolkit) is a **collection of Redux reducer logic and actions for a single feature of your application**, all defined in one file.

The name "slice" comes from the idea that your entire app's state is one big object, and a slice represents a **"slice" of that state** — one specific piece of it.

What a Slice Contains

A slice bundles three things together for one feature:

1. **State** — the initial data for that feature
2. **Reducers** — functions that define how the state changes
3. **Actions** — automatically generated based on your reducers

---

Simple Mental Model

Imagine your app's state as a pie 🥧:

```
App State (whole pie)
├── userSlice      🍕 (user login info)
├── cartSlice      🍕 (shopping cart items)
└── productsSlice  🍕 (list of products)
```

Each slice manages its own piece independently.

Code Example

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',            // name of this slice
  initialState: { value: 0 }, // its own piece of state
  reducers: {                  // functions to update state
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    reset:     (state) => { state.value = 0; },
  },
});

// Auto-generated actions
export const { increment, decrement, reset } = counterSlice.actions;

// The reducer to register in the store
export default counterSlice.reducer;
```

Why Slices Are Useful

Without Slice (old Redux)	With Slice (Redux Toolkit)
Actions defined separately	Actions auto-generated
Reducer in a different file	Reducer lives with the logic
Lots of boilerplate	Minimal, clean code
Easy to get out of sync	Everything stays together

One-Line Summary

> *A **slice** = the **state + reducers + actions** for **one feature**, kept together in one place using `createSlice()`.*

Think of it as a **self-contained module** for one part of your app's data.